

七、AI工程化与前端工具链（18题）

1. 如何设计一个AI前端项目的标准化目录结构

核心设计原则：模块化、关注点分离、可扩展性、易于维护。按功能领域组织代码，而非按技术类型。

目录结构设计：

```
代码块
1  ai-frontend/
2  |— src/
3  |   |— app/                # 应用级配置
4  |   |   |— layout/        # 布局组件
5  |   |   |— providers/     # 上下文提供者
6  |   |   |— routes/        # 路由配置
7  |   |   |— store/         # 全局状态
8  |   |— features/          # 功能模块（按业务领域划分）
9  |   |   |— chat/          # 聊天功能
10 |   |   |   |— api/        # API接口
11 |   |   |   |— components/ # 组件
12 |   |   |   |— hooks/      # 自定义Hook
13 |   |   |   |— store/      # 模块状态
14 |   |   |   |— types/      # 类型定义
15 |   |   |— code-generation/ # 代码生成
16 |   |   |— image-generation/ # 图像生成
17 |   |— shared/             # 共享资源
18 |   |   |— components/     # 通用UI组件
19 |   |   |— hooks/          # 通用Hook
20 |   |   |— utils/           # 工具函数
21 |   |   |— constants/      # 常量定义
22 |   |   |— configs/        # 配置
23 |   |— ai/                 # AI核心模块
24 |   |   |— models/         # 模型接口
25 |   |   |— prompts/        # 提示词模板
26 |   |   |— tools/          # AI工具函数
27 |   |   |— evaluators/     # 评估器
28 |   |— types/              # 全局类型定义
29 |   |— styles/             # 全局样式
30 |— tests/                  # 测试文件
31 |— public/                 # 静态资源
32 |— docs/                   # 项目文档
```

关键设计决策：

1. 按功能模块组织而非技术类型，便于功能扩展
 2. AI相关代码集中管理，便于模型切换和更新
 3. 严格区分业务逻辑和通用逻辑
 4. 测试文件靠近被测试代码，便于维护
-

2. 请设计一套AI前端代码规范

代码规范体系：建立编码规范、提交规范、命名规范三位一体的规范体系。

ESLint配置策略：

1. 基础规则：基于 `eslint:recommended`
2. TypeScript集成： `@typescript-eslint`
3. React规范： `eslint-plugin-react` 和 `eslint-plugin-react-hooks`
4. 导入排序： `eslint-plugin-import` 管理导入顺序
5. AI特定规则：自定义规则如禁止直接暴露API密钥

Prettier配置：

1. 格式统一：代码格式化规则
2. 集成ESLint：避免规则冲突
3. 提交前格式化：通过Git Hook自动格式化

Commitlint配置：

1. Conventional Commits： `<type>(<scope>): <description>`
2. 类型定义：feat、fix、docs、style、refactor、test、chore
3. AI相关类型：ai、model、prompt等自定义类型
4. 范围定义：chat、code、image等功能模块

Git Hooks集成：

1. pre-commit：运行ESLint和Prettier
 2. commit-msg：验证提交信息格式
 3. pre-push：运行核心测试
-

3. 如何打造AI项目的自动化提交与代码质量流水线

Husky配置：

- 1. 初始化Husky：自动安装Git Hooks
- 2. 配置Hook脚本：pre-commit、commit-msg、pre-push
- 3. 跨平台支持：处理Windows/macOS/Linux差异

lint-staged配置：

- 1. 按文件类型配置：对特定文件类型运行特定检查
- 2. 增量检查：只检查暂存区文件
- 3. 并行执行：多个检查器并行运行
- 4. AI代码特殊处理：对AI相关代码运行额外检查

Commitizen集成：

- 1. 自定义适配器：针对AI项目定制提交模板
- 2. 交互式提交：引导式提交信息填写
- 3. 范围选择：从预定义的功能模块中选择
- 4. 智能默认值：基于代码变更建议提交类型

自动化流水线：

代码块

```
1  开发 → git add → 触发pre-commit → ESLint检查 → Prettier格式化
2      → 通过 → git commit → 触发commit-msg → Commitlint验证
3      → 通过 → 提交完成 → git push → 触发pre-push → 运行核心测试
4      → 通过 → 推送远程
```

4. 设计一个AI前端项目的CI/CD流水线

CI/CD阶段设计：

阶段1：代码检查

- ESLint静态分析
- TypeScript类型检查
- 安全漏洞扫描
- 代码复杂度检查
- 重复代码检测

阶段2：单元测试

- 组件单元测试
- Hook测试
- 工具函数测试
- AI模型接口测试
- 测试覆盖率检查 (>80%)

阶段3：集成测试

- 组件集成测试
- 状态管理测试
- API集成测试
- AI服务集成测试

阶段4：E2E测试

- 用户流程测试
- AI交互测试
- 跨浏览器测试
- 性能基准测试
- 无障碍测试

阶段5：构建优化

- 代码分割优化
- 资源压缩
- 图片优化
- 缓存策略配置
- Source Map生成

阶段6：部署

- 测试环境部署
- 冒烟测试
- 性能测试
- 生产环境部署
- 健康检查

环境策略：

- 开发环境：每次提交自动部署
- 测试环境：主分支自动部署

- 预发布环境：手动触发，完整测试
 - 生产环境：蓝绿部署，渐进式发布
-

5. 如何用Docker容器化AI前端应用

Dockerfile设计：

多阶段构建：

1. 构建阶段：安装依赖，构建应用
2. 运行时阶段：最小化镜像，只包含运行所需文件

优化策略：

1. 依赖缓存：分层缓存依赖安装
2. 构建缓存：利用Docker构建缓存
3. 多架构支持：支持x86/ARM架构
4. 安全加固：非root用户运行，最小权限

容器编排：

1. 健康检查：应用健康检查端点
2. 就绪检查：依赖服务就绪检查
3. 资源限制：CPU/内存限制
4. 日志收集：结构化日志输出

开发环境：

1. 热重载：开发时容器内热重载
 2. 卷挂载：源代码卷挂载
 3. 网络配置：容器间网络通信
 4. 环境变量：开发/测试/生产环境配置
-

6. 请设计一个AI前端性能监控方案

核心Web指标监控：

1. FP/FCP：首次绘制/首次内容绘制
2. LCP：最大内容绘制
3. FID/INP：交互延迟
4. CLS：累积布局偏移

5. TTFB：首字节时间

AI特定指标：

1. 模型加载时间
2. Token生成速度
3. 流式响应延迟
4. 上下文窗口管理性能
5. 内存使用峰值

数据采集策略：

1. 真实用户监控：通过Performance API采集
2. 合成监控：定期自动化测试
3. 自定义指标：业务特定指标
4. 关联分析：关联性能与业务指标

可视化仪表板：

1. 实时监控：关键指标实时图表
2. 历史趋势：性能变化趋势
3. 异常检测：自动异常检测
4. 根因分析：性能问题根因定位

告警机制：

1. 阈值告警：指标超过阈值告警
 2. 趋势告警：性能下降趋势告警
 3. 智能基线：动态基线，自动适应
 4. 分级告警：P0-P4不同级别告警
-

7. 如何用Sentry监控AI前端异常

Sentry集成配置：

1. SDK配置：初始化配置，环境设置
2. 错误边界：React错误边界集成
3. 性能监控：性能数据上报
4. 会话回放：用户操作录制

AI上下文捕获：

1. 用户输入：捕获触发错误的用户输入
2. 模型参数：AI调用时的参数配置
3. 网络状态：请求时的网络状况
4. 设备信息：设备类型、内存、CPU
5. 应用状态：Redux/Zustand状态

错误分组策略：

1. 指纹识别：相似错误自动分组
2. 智能去重：避免重复告警
3. 错误趋势：错误频率趋势分析
4. 影响评估：影响用户数评估

workflow集成：

1. 通知集成：Slack、邮件、钉钉通知
2. 问题跟踪：Jira、GitHub Issues集成
3. 自动分配：基于规则自动分配负责人
4. 解决验证：错误解决后自动验证

8. 设计一个AI前端日志系统

日志架构设计：

日志级别：

1. ERROR：错误日志，需要立即处理
2. WARN：警告日志，需要注意
3. INFO：信息日志，业务关键操作
4. DEBUG：调试日志，开发调试
5. TRACE：追踪日志，详细追踪

结构化日志格式：

代码块

```
1  {
2    "timestamp": "ISO时间",
3    "level": "INFO",
4    "message": "描述信息",
5    "context": {
6      "userId": "用户ID",
```

```
7      "sessionId": "会话ID",
8      "requestId": "请求ID"
9    },
10    "data": { /* 业务数据 */ },
11    "metadata": {
12      "userAgent": "浏览器信息",
13      "url": "页面URL",
14      "environment": "环境"
15    }
16  }
```

日志分类：

1. 用户操作日志：用户交互记录
2. AI请求日志：AI API调用记录
3. 性能日志：性能指标记录
4. 错误日志：错误异常记录
5. 审计日志：安全审计记录

日志收集策略：

1. 客户端收集：浏览器中收集日志
2. 批量发送：日志批量发送，减少请求
3. 本地缓冲：网络异常时本地缓冲
4. 采样策略：生产环境采样，开发环境全量

日志分析：

1. 实时分析：实时日志流分析
2. 聚合分析：日志数据聚合统计
3. 关联分析：多日志源关联分析
4. 趋势分析：日志趋势变化分析

9. 如何用Webpack或Vite优化AI前端构建性能

Webpack优化策略：

代码分割：

1. 入口分割：多入口应用分割
2. 动态导入：路由级代码分割

3. 组件级分割：大组件异步加载
4. 第三方库分割：vendor chunk分离

Tree Shaking:

1. ES模块：使用ES模块语法
2. sideEffects：正确配置package.json
3. 作用域提升：ModuleConcatenationPlugin
4. 深度分析：深层依赖分析

缓存优化:

1. 持久缓存：contenthash文件名
2. 模块缓存：HardSourceWebpackPlugin
3. 构建缓存：cache-loader
4. 资源配置：resource hint预加载

资源优化:

1. 图片压缩：image-webpack-loader
2. 字体子集：fontmin-webpack
3. CSS提取：MiniCssExtractPlugin
4. 资源内联：小资源内联

Vite优化策略:

1. 依赖预构建：优化第三方依赖
2. 按需编译：开发时按需编译
3. 快速HMR：热模块替换优化
4. 构建优化：Rollup构建优化

AI特定优化:

1. 模型文件：WASM模型文件特殊处理
2. 大文件：大资源分块加载
3. 预加载：AI功能预加载
4. 运行时优化：AI运行时性能优化

10. 请设计一个AI前端依赖管理策略

依赖管理流程:

依赖分类：

- 1. 核心依赖：React、TypeScript、状态管理
- 2. UI依赖：组件库、图标库
- 3. AI依赖：模型SDK、AI工具库
- 4. 工具依赖：构建工具、测试工具
- 5. 开发依赖：开发辅助工具

更新策略：

- 1. 安全更新：安全漏洞立即更新
- 2. 小版本更新：自动更新，自动测试
- 3. 大版本更新：手动评估，分步更新
- 4. 破坏性更新：完整测试，迁移计划

兼容性评估：

- 1. API兼容性：API变化评估
- 2. 性能影响：更新后性能测试
- 3. 包大小：包体积变化评估
- 4. 依赖冲突：依赖版本冲突解决

自动化管理：

- 1. 自动检查：定期检查过时依赖
- 2. 自动测试：依赖更新后自动测试
- 3. 自动创建PR：自动创建更新PR
- 4. 自动回滚：测试失败自动回滚

AI SDK管理：

- 1. 版本锁定：AI SDK版本严格锁定
- 2. 回退计划：新版问题快速回退
- 3. 多版本支持：支持多版本并行
- 4. 迁移工具：版本迁移辅助工具

11. 设计一个AI前端错误上报与告警系统

错误收集架构：

错误分类：

- 1. JavaScript错误：运行时错误
- 2. 资源错误：加载失败错误
- 3. Promise错误：未捕获Promise错误
- 4. 网络错误：API请求错误
- 5. AI错误：AI服务相关错误

错误上下文：

- 1. 用户信息：用户ID、角色
- 2. 会话信息：会话ID、操作序列
- 3. 设备信息：设备类型、系统版本
- 4. 网络信息：网络类型、信号强度
- 5. 应用状态：当前路由、状态数据

告警规则：

- 1. 频率告警：短时间内相同错误多次
- 2. 影响面告警：影响用户数超过阈值
- 3. 严重性告警：关键功能错误
- 4. 趋势告警：错误率上升趋势
- 5. 组合告警：多个相关错误同时发生

通知渠道：

- 1. 即时通讯：Slack、钉钉、飞书
- 2. 邮件通知：分级邮件通知
- 3. 短信通知：P0级严重错误
- 4. 电话通知：系统级故障
- 5. 仪表板：统一监控仪表板

告警处理流程：

- 1. 自动分派：基于规则自动分配
 - 2. 升级策略：超时未处理自动升级
 - 3. 处理跟踪：错误处理状态跟踪
 - 4. 解决验证：问题解决后验证
 - 5. 根本原因分析：错误根因分析
-

12. 如何用GraphQL Code Generator自动生成TypeScript类型

集成流程：

GraphQL Schema管理：

1. Schema定义：定义完整的GraphQL Schema
2. Schema校验：Schema语法和逻辑校验
3. Schema发布：版本化发布Schema
4. Schema文档：自动生成API文档

Code Generator配置：

1. 配置文件：codegen.yml配置生成规则
2. 类型生成：生成TypeScript类型定义
3. React Hooks：生成React Query/Apollo Hooks
4. 操作生成：生成查询/变更操作

开发体验优化：

1. 实时生成：开发时监听Schema变化
2. 类型安全：完整类型提示
3. 自动补全：IDE自动补全支持
4. 重构友好：类型安全的重构

CI集成：

1. 生成检查：CI中验证类型同步
2. 差异检测：检测未提交的类型文件
3. 自动化成：Schema更新后自动生成
4. 版本管理：类型文件版本管理

AI特定扩展：

1. AI操作类型：AI相关操作类型定义
2. 流式响应类型：流式响应类型生成
3. 模型参数类型：模型参数类型定义
4. 工具调用类型：AI工具调用类型

13. 请设计一个AI前端配置管理系统

配置架构设计：

配置分类：

1. 环境配置：开发、测试、生产环境
2. 功能开关：功能级别开关控制
3. AI配置：模型端点、参数、提示词
4. 业务配置：业务规则、阈值
5. 界面配置：主题、布局、语言

配置存储：

1. 环境变量：基础环境配置
2. 配置文件：JSON/TypeScript配置文件
3. 远程配置：配置服务动态下发
4. 本地存储：用户偏好配置

动态配置：

1. 热更新：运行时配置更新
2. 条件配置：基于条件的配置
3. 用户配置：用户级配置覆盖
4. 实验配置：A/B测试配置

配置管理：

1. 版本控制：配置变更版本管理
2. 回滚能力：配置错误快速回滚
3. 审计日志：配置变更审计
4. 权限控制：配置修改权限控制

AI配置管理：

1. 模型配置：模型版本、参数配置
2. 提示词管理：提示词模板管理
3. 工具配置：AI工具配置管理
4. 实验配置：AI实验参数配置

配置验证：

1. Schema验证：配置结构验证
2. 值验证：配置值范围验证
3. 依赖验证：配置间依赖验证
4. 运行时验证：运行时配置验证

14. 如何用Turborepo或Nx管理AI Monorepo

Monorepo架构设计：

工作空间配置：

1. 包管理：workspace协议管理内部依赖
2. 任务定义：定义构建、测试、发布任务
3. 依赖图：可视化包依赖关系
4. 影响分析：变更影响范围分析

构建缓存：

1. 本地缓存：开发构建缓存
2. 远程缓存：CI/CD共享缓存
3. 缓存策略：按文件哈希缓存
4. 缓存失效：依赖变更自动失效

任务调度：

1. 并行执行：独立任务并行执行
2. 依赖感知：基于依赖图执行任务
3. 增量构建：只构建变更部分
4. 资源限制：并发任务数限制

依赖图优化：

1. 循环依赖检测：自动检测循环依赖
2. 无用依赖移除：自动移除未使用依赖
3. 依赖版本统一：统一依赖版本
4. 依赖分析：依赖大小、性能分析

AI包管理：

1. 模型包：AI模型封装包
2. 工具包：AI工具函数包
3. 客户端包：AI服务客户端
4. 类型包：共享类型定义包

开发体验：

1. 项目生成：脚手架生成新包

2. 代码共享：跨包代码共享
 3. 文档生成：自动生成包文档
 4. 发布管理：统一版本管理和发布
-

15. 设计一个AI前端文档站点

文档体系架构：

文档分类：

1. 快速开始：新手上手指南
2. API文档：完整API参考
3. 组件文档：UI组件使用文档
4. AI指南：AI功能使用指南
5. 最佳实践：开发最佳实践
6. 故障排查：常见问题解决
7. 版本历史：变更日志

文档工具：

1. 文档生成：TypeDoc、Storybook
2. 站点生成：Docusaurus、VitePress
3. 交互示例：可运行代码示例
4. 搜索功能：全文搜索支持

内容管理：

1. 版本化文档：多版本文档支持
2. 多语言：国际化文档支持
3. 权限控制：内部/公开文档分离
4. 贡献指南：文档贡献指南

AI文档特色：

1. 提示词库：可搜索的提示词库
2. 模型比较：不同模型能力对比
3. 成本计算：API调用成本计算
4. 性能基准：性能基准测试结果

集成与部署：

1. CI集成：文档自动构建
 2. 预览环境：PR文档预览
 3. 搜索优化：SEO优化
 4. 访问统计：文档访问分析
-

16. 如何管理AI项目版本号、生成变更日志、自动发布NPM包

版本管理策略：

版本号规范：

1. 语义化版本：major.minor.patch
2. 预发布版本：alpha、beta、rc
3. AI版本：模型版本集成
4. 兼容性标识：破坏性变更标识

Changesets工作流：

1. 变更描述：开发者提交变更描述
2. 版本计算：自动计算版本号
3. 变更日志：自动生成CHANGELOG
4. 发布流水线：自动发布流程

发布策略：

1. 自动发布：满足条件自动发布
2. 手动审批：重大变更手动审批
3. 渐进发布：金丝雀发布策略
4. 回滚计划：发布问题快速回滚

NPM发布：

1. 自动构建：发布前自动构建
2. 版本标记：latest、next、stable标签
3. 发布检查：发布前完整性检查
4. 发布通知：发布成功通知

AI包发布：

1. 模型包：AI模型独立发布
2. 客户端包：AI客户端SDK发布

- 3. 工具包：AI工具库发布
- 4. 模板包：提示词模板发布

质量保证：

- 1. 发布检查清单：发布前检查项
 - 2. 自动化测试：发布前自动化测试
 - 3. 兼容性测试：向后兼容性测试
 - 4. 文档同步：文档自动更新
-

17. 如何用Bundle Analyzer分析AI前端打包体积

打包分析流程：

分析工具配置：

- 1. Webpack Bundle Analyzer：生成分析报告
- 2. Source Map Explorer：源码映射分析
- 3. Webpack Stats：构建统计数据分析
- 4. 自定义分析：AI特定包分析

分析维度：

- 1. 总体积：应用总体积分析
- 2. 初始加载：初始包体积分析
- 3. 异步加载：异步包体积分析
- 4. 重复代码：重复代码检测
- 5. 未使用代码：未使用代码检测

AI包分析：

- 1. 模型SDK：AI SDK体积分析
- 2. 可视化库：图表库体积分析
- 3. 工具库：工具函数体积分析
- 4. 多语言：国际化包体积

优化策略：

- 1. 代码分割：按需加载优化
- 2. 依赖优化：替换大体积依赖
- 3. 树摇优化：增强Tree Shaking

4. 压缩优化：高级压缩配置
5. 运行时加载：运行时动态加载

监控与告警：

1. 体积监控：包体积变化监控
2. 阈值告警：体积超过阈值告警
3. 趋势分析：体积增长趋势分析
4. 回归检测：体积回归自动检测

CI集成：

1. 自动化分析：CI中自动运行分析
 2. 报告生成：生成分析报告
 3. 差异对比：与基准对比分析
 4. 质量门禁：体积检查门禁
-

18. 设计一个AI前端灰度发布方案

灰度发布架构：

发布策略：

1. 用户分群：基于用户属性分群
2. 百分比发布：按百分比逐步放量
3. 功能开关：功能级别开关控制
4. 渐进式发布：逐步扩大范围

用户分群规则：

1. 用户ID：基于用户ID哈希
2. 设备类型：iOS/Android/Web
3. 地理位置：地区/国家/城市
4. 用户标签：新用户/老用户/付费用户
5. 自定义规则：业务自定义规则

发布流程：

1. 内测阶段：内部员工测试
2. Alpha阶段：小范围用户测试
3. Beta阶段：中等范围用户测试

4. 全面发布：全量用户发布

5. 观察期：发布后观察期

监控与回滚：

1. 核心指标监控：错误率、性能指标

2. 业务指标监控：转化率、留存率

3. 用户反馈：用户反馈收集

4. 自动回滚：指标异常自动回滚

5. 手动回滚：手动触发回滚

AI功能发布：

1. 模型切换：新旧模型并行运行

2. 结果对比：新旧结果对比分析

3. 质量评估：AI回答质量评估

4. 成本监控：API成本变化监控

发布管理：

1. 发布计划：详细的发布计划

2. 沟通机制：变更沟通机制

3. 应急计划：应急响应计划

4. 事后复盘：发布后复盘分析